Assignment 3 — bộ đệm vòng (circular buffer)

Bản sinh viên.

---

1. Giới thiệu và quy định nộp bài

Trong các hệ thống nhúng thực tế (như thu thập mẫu cảm biến ADC, nhận dữ liệu qua cổng nối tiếp UART/SPI/CAN giữa trình phục vụ ngắt ISR và vòng lặp chính), **bộ đệm vòng (circular buffer / ring buffer)** là cấu trúc dữ liệu nền tảng giúp truyền dữ liệu theo cơ chế **vào trước – ra trước (FIFO)** trên vùng nhớ tĩnh cố định mà không cần cấp phát động hay dịch chuyển phần tử.

Bài tập này yêu cầu các em lập trình bộ đệm vòng theo hai phương pháp cốt lõi mà **không sử dụng thêm biến đếm số phần tử (`count`)**:

- **Bài 1 (4.0 điểm độ chính xác):** cài đặt bộ đệm vòng chỉ sử dụng **mảng cố định và 2 biến chỉ số** (`head`, `tail`).
- **Bài 2 (4.0 điểm độ chính xác):** cài đặt bộ đệm vòng chỉ sử dụng **mảng cố định và 2 con trỏ** (`head`, `tail` trỏ trực tiếp vào các ô nhớ của mảng).

Ngoài 8.0 điểm độ chính xác của hai bài, **2.0 điểm được chấm chung cho coding style và độ tối ưu của mã nguồn** (chi tiết tại mục 5).

Đi kèm tài liệu này gồm có:

1. **`bai_tap.h`**: khai báo cấu trúc dữ liệu và nguyên mẫu các hàm cần hoàn thiện.
2. **`bai_tap.cpp`**: tệp mã nguồn khung chứa các hàm của bài 1, bài 2 để sinh viên điền mã nguồn, đồng thời **tích hợp sẵn khung mô phỏng kịch bản ghi – đọc (*writer–reader scenario*) và hàm `main()`** ở cuối tệp để chạy thử ngay khi làm bài.

Quy cách làm bài và nộp bài:

1. Sinh viên giữ nguyên tệp `bai_tap.h` và hoàn thiện toàn bộ các hàm của bài 1 và bài 2 ngay trong tệp **`bai_tap.cpp`**.
2. Sinh viên có thể biên dịch và chạy trực tiếp `bai_tap.cpp` để kiểm thử kịch bản *writer–reader* và bổ sung thêm các ca kiểm thử riêng trong hàm `student_custom_tests()`:

   ```
   c++ -std=c++20 -Wall -Wextra -Wpedantic bai_tap.cpp -o bai_tap
   ./bai_tap
   ```

3. **Tệp cần nộp:** nén tệp **`bai_tap.cpp`** đã hoàn thiện thành một tệp ZIP có tên **`assignment3_{student_id}.zip`**, trong đó `{student_id}` được thay bằng mã số sinh viên. Ví dụ: sinh viên có mã số `20241234` nộp tệp **`assignment3_20241234.zip`**; không ghi dấu ngoặc nhọn trong tên tệp thực tế.
4. **Nội dung ZIP:** chỉ chứa **`bai_tap.cpp`** ngay ở thư mục gốc của tệp ZIP (có thể giữ nguyên phần mô phỏng và hàm `main()` ở cuối tệp). Không đưa `bai_tap.h`, tệp thực thi hoặc thư mục trung gian vào ZIP.

---

2. Nguyên lý chung: phân biệt rỗng và đầy khi chỉ dùng 2 vị trí (head, `tail`)

Kích thước vật lý của mảng lưu trữ được ấn định bởi hằng số:

```
constexpr int CB_CAPACITY = 8;
```

Khi cấu trúc bộ đệm vòng **chỉ lưu mảng và 2 vị trí head (ghi) / `tail` (đọc)** mà không có biến `count`:

- Nếu cho phép ghi kín cả 8 ô nhớ, trạng thái đầy cũng dẫn đến `head == tail` giống hệt trạng thái rỗng, gây nhập nhằng không thể phân biệt.
- **Giải pháp tiêu chuẩn trong lập trình nhúng:** luôn giữ lại **1 ô nhớ trống** làm ranh giới ngăn cách.
  - **Trạng thái rỗng (empty):** `head == tail` (số phần tử đang lưu bằng `0`).
  - **Trạng thái đầy (full):** vị trí kế tiếp (sau khi tính xoay vòng) của `head` chạm vào `tail`. Khi đó bộ đệm đang chứa tối đa **`CB_CAPACITY - 1` = 7 phần tử**, và mọi thao tác ghi thêm phải bị từ chối (`return false`) để bảo vệ dữ liệu chưa đọc.

---

3. Mô tả chi tiết các bài tập

Bài 1 — bộ đệm vòng dùng mảng và 2 chỉ số (array & 2 indices) (4.0 điểm độ chính xác)

Cấu trúc dữ liệu và nguyên mẫu hàm

```
struct IndexCircularBuffer {
    int data[CB_CAPACITY];
    int head; /* Chỉ số ghi (writer index): 0 .. CB_CAPACITY - 1 */
    int tail; /* Chỉ số đọc (reader index): 0 .. CB_CAPACITY - 1 */
};

void cb_index_init(IndexCircularBuffer& cb);
bool cb_index_is_empty(const IndexCircularBuffer& cb);
bool cb_index_is_full(const IndexCircularBuffer& cb);
int  cb_index_size(const IndexCircularBuffer& cb);
bool cb_index_write(IndexCircularBuffer& cb, int value);
bool cb_index_read(IndexCircularBuffer& cb, int& out_value);
```

Đặc tả chi tiết từng hàm

1. **`cb_index_init(cb)`**: đặt bộ đệm về trạng thái rỗng ban đầu với `head = 0` và `tail = 0`.
2. **`cb_index_is_empty(cb)`**: trả về `true` nếu bộ đệm không có phần tử nào (`head == tail`), ngược lại trả về `false`.
3. **`cb_index_is_full(cb)`**: trả về `true` nếu bước tiến tiếp theo của `head` chạm `tail`, tức là `(cb.head + 1) % CB_CAPACITY == cb.tail`.
4. **`cb_index_size(cb)`**: trả về số lượng phần tử hiện có trong bộ đệm (giá trị từ `0` đến `CB_CAPACITY - 1`, tức `0..7`).
   - *Lưu ý:* khi `head` đã xoay vòng về đầu mảng và nhỏ hơn `tail` (ví dụ `tail = 5`, `head = 2`), số phần tử hiện có là `(2 - 5 + 8) % 8 = 5`.
5. **`cb_index_write(cb, value)`**:

- Nếu bộ đệm đã đầy (`cb_index_is_full(cb)` là `true`): không ghi vào mảng, giữ nguyên `head` và trả về `false`.
- Nếu chưa đầy: lưu `value` vào `cb.data[cb.head]`, tăng `cb.head` lên 1 theo quy tắc xoay vòng `(cb.head + 1) % CB_CAPACITY`, và trả về `true`.

6. **`cb_index_read(cb, out_value)`**:
   - Nếu bộ đệm đang rỗng (`cb_index_is_empty(cb)` là `true`): không thay đổi `out_value`, giữ nguyên `tail` và trả về `false`.
   - Nếu có dữ liệu: gán `out_value = cb.data[cb.tail]`, tăng `cb.tail` lên 1 theo quy tắc xoay vòng `(cb.tail + 1) % CB_CAPACITY`, và trả về `true`.

Ví dụ minh họa từng bước (với `CB_CAPACITY = 8`)

Bước thao tác	Giá trị trả về	`head`	`tail`	`size`	`empty`	`full`	Giải thích
`cb_index_init(cb)`	—	`0`	`0`	`0`	`true`	`false`	Khởi tạo bộ đệm rỗng
`cb_index_read(cb, x)`	`false`	`0`	`0`	`0`	`true`	`false`	Đọc khi rỗng bị từ chối, `x` giữ nguyên
Ghi 3 số: `10`, `20`, `30`	`true` (x3)	`3`	`0`	`3`	`false`	`false`	Lưu tại chỉ số `0, 1, 2`; `head` tiến lên `3`
Đọc 2 lần vào `x`	`true` (`10`, `20`)	`3`	`2`	`1`	`false`	`false`	Lấy ra `10, 20`; `tail` tiến lên `2`, còn số `30`
Ghi 6 số: `40, 50, 60, 70, 80, 90`	`true` (x6)	`1`	`2`	`7`	`false`	`true`	`head` đi qua `3,4,5,6,7,0` rồi dừng ở `1`. Vì `(1+1)%8 == 2 == tail` nên đầy
`cb_index_write(cb, 99)`	`false`	`1`	`2`	`7`	`false`	`true`	Bộ đệm đã đầy (7 phần tử), từ chối ghi `99`

Bài 2 — bộ đệm vòng dùng mảng và 2 con trỏ (array & 2 pointers) (4.0 điểm độ chính xác)

Bối cảnh bài toán

Trong nhiều trình điều khiển thiết bị (driver) cấp thấp trên vi điều khiển, thay vì giữ chỉ số nguyên rồi mỗi lần truy cập phải tính địa chỉ `base + index * sizeof(type)`, người ta sử dụng trực tiếp **hai con trỏ `head` và `tail` trỏ thẳng vào các ô nhớ trong mảng `data`**. Khi con trỏ tiến thêm 1 phần tử (`++ptr`) và chạm tới địa chỉ ngay sau phần tử cuối cùng của mảng (`cb.data + CB_CAPACITY`), con trỏ được gán quay về địa chỉ đầu mảng (`cb.data`).

Cấu trúc dữ liệu và nguyên mẫu hàm

```
struct PointerCircularBuffer {
    int data[CB_CAPACITY];
    int* head; /* Con trỏ ghi (writer pointer) */
    int* tail; /* Con trỏ đọc (reader pointer) */
};

void cb_ptr_init(PointerCircularBuffer& cb);
bool cb_ptr_is_empty(const PointerCircularBuffer& cb);
bool cb_ptr_is_full(const PointerCircularBuffer& cb);
int  cb_ptr_size(const PointerCircularBuffer& cb);
bool cb_ptr_write(PointerCircularBuffer& cb, int value);
bool cb_ptr_read(PointerCircularBuffer& cb, int& out_value);
```

Đặc tả chi tiết từng hàm

1. **`cb_ptr_init(cb)`**: cho cả hai con trỏ `cb.head` và `cb.tail` cùng trỏ vào phần tử đầu tiên của mảng (`cb.data`, tức `&cb.data[0]`).
2. **`cb_ptr_is_empty(cb)`**: trả về `true` nếu hai con trỏ trỏ vào cùng một địa chỉ (`cb.head == cb.tail`).
3. **`cb_ptr_is_full(cb)`**: xác định địa chỉ ô nhớ kế tiếp của `cb.head` (nếu `cb.head + 1 == cb.data + CB_CAPACITY` thì địa chỉ kế tiếp là `cb.data`, ngược lại là `cb.head + 1`). Nếu địa chỉ kế tiếp này bằng `cb.tail` thì trả về `true`.
4. **`cb_ptr_size(cb)`**: tính số phần tử hiện có dựa trên hiệu hai con trỏ (`cb.head − cb.tail`):
   - Nếu `cb.head >= cb.tail`: số phần tử là `static_cast<int>(cb.head − cb.tail)`.
   - Nếu `cb.head < cb.tail` (khi head đã xoay vòng về trước `tail`): số phần tử là `static_cast<int>((cb.head − cb.tail) + CB_CAPACITY)`.
5. **`cb_ptr_write(cb, value)`**:
   - Nếu bộ đệm đã đầy: không ghi dữ liệu, giữ nguyên `cb.head` và trả về `false`.
   - Nếu chưa đầy: giải tham chiếu `*cb.head = value`, dịch chuyển `cb.head` sang ô kế tiếp (xoay vòng về `cb.data` nếu chạm `cb.data + CB_CAPACITY`) và trả về `true`.
6. **`cb_ptr_read(cb, out_value)`**:
   - Nếu bộ đệm đang rỗng: giữ nguyên `out_value`, giữ nguyên `cb.tail` và trả về `false`.
   - Nếu có dữ liệu: đọc `out_value = *cb.tail`, dịch chuyển `cb.tail` sang ô kế tiếp (xoay vòng về `cb.data` nếu chạm `cb.data + CB_CAPACITY`) và trả về `true`.

Ví dụ minh họa với địa chỉ con trỏ

Giả sử mảng `cb.data` gồm 8 phần tử từ `&data[0]` đến `&data[7]`, mốc kết thúc mảng là `end = cb.data + 8` (`&data[8]` — địa chỉ ngay sau phần tử cuối, chỉ dùng để so sánh mốc xoay vòng, tuyệt đối không giải tham chiếu):

- **Khởi tạo:** `head = &data[0]`, `tail = &data[0]` → `head == tail` (rỗng, `size = 0`).
- **Ghi 7 phần tử:** `head` lần lượt ghi vào `data[0] .. data[6]` và dừng tại `&data[7]`. Lúc này ô kế tiếp của `head` là `&data[7] + 1 = end` → xoay vòng thành `&data[0] == tail` → bộ đệm **đầy** (`size = 7`).
- **Đọc 3 phần tử:** `tail` tiến từ `&data[0]` lên `&data[3]`. Lúc này `size = (&data[7] − &data[3]) = 4`.
- **Ghi thêm 2 phần tử:**
  - Phần tử thứ nhất ghi vào `*head` (`data[7]`), sau đó `head` tăng lên `end` nên xoay vòng về `&data[0]`.
  - Phần tử thứ hai ghi vào `*head` (`data[0]`), sau đó `head` tiến lên `&data[1]`.
  - Lúc này `head = &data[1] < tail = &data[3]`, số phần tử trong đệm là `(&data[1] − &data[3]) + 8 = −2 + 8 = 6`.

---

4. Kịch bản ghi – đọc (writer–reader scenario) và các ca kiểm thử cần lưu ý

Ở cuối tệp `bai_tap.cpp` đã tích hợp sẵn khung mô phỏng kịch bản thực tế cùng hàm `main()`:

- **Bên ghi (writer):** đóng vai trò ngắt ngoại vi đẩy các lô dữ liệu vào bộ đệm.
- **Bên đọc (reader):** đóng vai trò vòng lặp xử lý lấy dữ liệu ra với tốc độ khác với bên ghi.

Khi tự kiểm thử chương trình của mình, sinh viên cần suy nghĩ và kiểm chứng đầy đủ **10 tình huống biên (test cases)** sau đây cho cả bài 1 và bài 2:

1. **Khởi tạo chuẩn (initialization):** ngay sau `init`, bộ đệm phải báo `is_empty() == true`, `is_full() == false`, `size() == 0`.
2. **Đọc khi bộ đệm rỗng (underflow):** gọi `read` ngay khi bộ đệm đang rỗng phải trả về `false`, biến `out_value` truyền vào phải giữ nguyên giá trị ban đầu (không bị ghi đè bởi dữ liệu rác), và vị trí `head`/`tail` không bị dịch chuyển.
3. **Ghi và đọc 1 phần tử đơn lẻ:** ghi 1 phần tử rồi đọc ra ngay: giá trị đọc ra phải khớp chính xác, bộ đệm trở lại trạng thái rỗng nhưng `head` và `tail` lúc này cùng nằm ở vị trí `1` (`&data[1]`).
4. **Bảo toàn thứ tự FIFO:** ghi một dãy số bất kỳ (gồm cả số dương, số âm và số `0`) rồi đọc ra tuần tự, kết quả phải đúng thứ tự đã ghi.
5. **Ghi chạm ngưỡng đầy tối đa (`CB_CAPACITY – 1` phần tử):** với `CB_CAPACITY = 8`, ghi liên tiếp 7 phần tử phải thành công cả 7 lần, sau phần tử thứ 7 thì `is_full() == true` và `size() == 7`.
6. **Ghi khi bộ đệm đã đầy (overflow protection):** khi bộ đệm đang đầy 7 phần tử, cố tình gọi `write` lần thứ 8 phải trả về `false`. Quan trọng hơn: 7 phần tử đang nằm trong bộ đệm không được phép bị ghi đè hay mất mát.
7. **Xoay vòng qua ranh giới cuối mảng (wrap-around):** ghi 5 phần tử, đọc 4 phần tử, sau đó ghi tiếp 5 phần tử để bắt buộc `head` (và sau đó là `tail`) vượt qua ô cuối `data[7]` và quay về `data[0]`.
8. **Tính `size()` khi `head` nằm trước `tail`:** khi đã xoay vòng khiến vị trí của `head` nhỏ hơn `tail`, hàm `size()` phải trả về số dương chính xác thay vì số âm.
9. **Đầy và rỗng tại vị trí bất kỳ giữa mảng:** khi `tail` đang nằm ở vị trí giữa mảng (ví dụ ô thứ 4), ghi đủ 7 phần tử khiến `head` xoay vòng bám sát ngay sau `tail` (ô thứ 3) → phải nhận diện đúng trạng thái đầy; khi đọc hết cả 7 phần tử để `tail` đuổi kịp `head` tại ô thứ 3 → phải nhận diện đúng trạng thái rỗng.
10. **Chu kỳ ghi – đọc liên tục nhiều vòng (stress / multi-cycle writer–reader):** cho bên ghi ghi từng đợt 3 phần tử và bên đọc đọc từng đợt 2 phần tử lặp lại qua hàng chục vòng quay của mảng để đảm bảo con trỏ/chỉ số không bao giờ trượt ra ngoài phạm vi mảng (`out-of-bounds`).

---

5. Tiêu chí đánh giá

Bài tập được đánh giá theo thang **10.0 điểm** với các đầu mục chính sau:

Tiêu chí	Điểm tối đa
Độ chính xác của thuật toán và chương trình	8.0
Độ tối ưu của thuật toán và mã nguồn	1.0
Coding style	1.0
Tổng	**10.0**

Phần độ chính xác được chia đều cho bài 1 và bài 2, mỗi bài tối đa **4.0 điểm**. Độ tối ưu và coding style được đánh giá chung trên mã nguồn của cả hai bài.

File ca kiểm thử tự động dùng để chấm điểm không được công bố.